

# EE/CS 148B HW 3

## Vision-Language Models

Lead TAs: Aadarsh Sahoo & Ziqi Ma

Spring 2026

### 1 Assignment Overview

In this assignment, you will build a Vision-Language Model (VLM) from the ground up. You will first implement a Vision Transformer (ViT), pretrain it CLIP-style on a new domain, adapt it efficiently with LoRA, and finally fuse it with a pretrained decoder language model to produce a VLM capable of answering questions about images. Along the way, you will explore architectural design choices and positional encoding schemes (including RoPE and its multimodal extensions) that matter in practice.

#### What you will implement.

1. A Vision Transformer image encoder (§2)
2. CLIP-style contrastive pretraining on EuroSAT (§3)
3. LoRA adapters from scratch and a comparison to full fine-tuning (§4)
4. A Vision-Language Model via fusion with a pretrained decoder (§5)
5. Rotary Position Embeddings (RoPE) and its 2D / multimodal extensions (§6)

**What the code looks like.** All the assignment code is hosted on GitHub at <https://github.com/caltech-eecs148b/hw3>. For details on the repository layout, please refer to the `README.md` file in the repository. The starter code provides Transformer building blocks (attention head, multi-head attention, MLP, and a generic `Block` with an `is_decoder` flag), along with dataset loaders, evaluation utilities, and test hooks. Each implementation section below starts with a **Files to update** box; use those boxes as the authoritative map from assignment problems to starter-code files.

#### Quick file map.

- §2: `basics/vit.py`
- §3: `vlm/clip.py`, `scripts/pretrain_clip.py`
- §4: `basics/lora.py`, `scripts/finetune_resisc.py`
- §5: `vlm/projector.py`, `vlm/model.py`, `scripts/train_vlm.py`, `scripts/eval_vlm.py`
- §6: `basics/rope.py`; also `basics/vit.py` and `vlm/model.py` for the RoPE ablations and bonus

**Where to get the data.** This assignment uses three datasets:

- **EuroSAT** (Sentinel-2 satellite imagery, 10 land-use classes, 27k images). Load via HuggingFace datasets: [https://huggingface.co/datasets/blanchon/EuroSAT\\_RGB](https://huggingface.co/datasets/blanchon/EuroSAT_RGB).
- **RESISC45** (remote-sensing scene classification, 45 classes). Load via HuggingFace datasets: <https://huggingface.co/datasets/timm/resisc45>.
- **CLEVR** (compositional visual question answering). We use a preprocessed 10k-example subset with the original CLEVR image resolution. The starter code expects it at `data/clevr_mini/`; run `uv run python scripts/download_clevr.py` to download the zip from Google Drive and extract it. Manual link: <https://drive.google.com/file/d/1KsswLqfYL1d91pg5kGUgwtPslo8njTB/view?usp=sharing>.

**How to submit.** You will submit the following files to Gradescope:

- **writeup.pdf**: Answer all the written questions. Please typeset your responses.
- **code.zip**: Contains all the code you have written.

**Note on Compute.** We assume students have access to Colab Pro+, which provides access to A100 and H100 GPUs. You can swap the GPU in Colab via “Runtime” → “Change Runtime Type”. This plan comes with “compute units” and allocations, so we recommend the following usage:

- For the ViT (§2) and the LoRA benchmarking experiments (§4), the “free” tier GPUs (T4, L4) should be sufficient. These analyses are not compute intensive and a correct implementation should work on these machines.
- For CLIP-style pretraining (§3) we recommend an L4 or A100 GPU.
- For VLM training (§5) and the RoPE ablations (§6), use an A100 or H100 GPU. We have tested the implementation on both and each training run should complete in around one hour. Accessing an A100 should be more reliable than an H100, as H100s are more limited.

You may use the same Colab Pro+ subscription from Homework 2; we DO NOT ask you to resubscribe or buy additional compute units for this homework. We will be reimbursing students for subscriptions to Google Colab Pro+ at the end of the quarter. Please keep your receipts for the reimbursement process. Alternatively, if you have other means of compute that you would like to use, feel free to.

## 2 Vision Transformer

In this section, you will build a Vision Transformer (ViT) [1], the image encoder we will use for the rest of the assignment. A ViT operates on an image by (1) splitting it into non-overlapping patches, (2) linearly projecting each patch into a fixed-dimensional embedding, (3) prepending a learnable [CLS] token and adding positional embeddings, and (4) processing the resulting sequence through a stack of standard Transformer blocks. It is, in essence, a Transformer that treats image patches the way a language model treats word tokens.

## Files to update for this section

Implement `PatchEmbeddings` and ViT in `basics/vit.py`. The public tests call your code through `tests/adapters.py`; the default adapter already imports `basics.vit`, so you should only need to edit adapter hooks if you intentionally change how tests bind to your implementation.

## 2.1 Setup — Importing the Basics Transformer Model

The starter code provides the Transformer building blocks (`Head`, `MultiHeadAttention`, `MLP`, `Block`) in the `basics` package. Each block takes an `is_decoder` flag that toggles the causal attention mask; for a ViT we want non-causal (bidirectional) attention within the image, so we will always pass `is_decoder=False`. By calling `uv run [command]` as usual, `uv` will automatically locate this local `basics` package. You can test that you can import the building blocks with:

```
~$ uv run python
Using CPython 3.12.10
Creating virtual environment at: /path/to/uv/env/dir
...
Installed 85 packages in 711ms
Python 3.12.10 (main, Apr 9 2025, 04:03:51) [Clang 20.1.0 ] on linux
...
>>> import basics
>>> from basics.model import Block, MultiHeadAttention
>>>
```

## 2.2 Patch Embeddings

The first step of the ViT is to split an input image of shape  $[B, 3, H, W]$  into a sequence of patch embeddings of shape  $[B, N, d_{\text{model}}]$ , where  $N = (H/P)^2$  is the number of patches and  $P$  is the patch size. A clean way to implement this is with a strided `Conv2d` whose kernel size and stride both equal  $P$ : each stride step extracts a non-overlapping patch and linearly projects it into  $d_{\text{model}}$  dimensions.

### Problem (patch\_embeddings): Implementing patchification

**File to update:** `basics/vit.py`.

**Deliverable:** Implement a `PatchEmbeddings` module with the following signature:

```
class PatchEmbeddings(nn.Module):
    def __init__(self, img_size: int, patch_size: int, d_model: int):
        ...

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (B, 3, img_size, img_size) float tensor.
        Returns:
            (B, num_patches, d_model) float tensor.
```

```
"""
```

To test your code, run the test with `uv run pytest -k test_patch_embeddings` and make sure your implementation passes it.

## 2.3 Assembling the ViT

We now combine patch embeddings with a learnable [CLS] token, learnable positional embeddings, and a stack of Transformer blocks. The output of the ViT will be the embedding corresponding to the [CLS] token after the final block, which we will use as a summary representation of the image.

### Problem (vit): Building the ViT

**File to update:** `basics/vit.py`.

**Deliverable:** Implement a ViT module with the following signature:

```
class ViT(nn.Module):
    def __init__(
        self,
        img_size: int,
        patch_size: int,
        d_model: int,
        num_heads: int,
        num_blocks: int,
        dropout: float = 0.1,
    ):
        ...

    def forward(self, x: torch.Tensor, return_all_tokens: bool = False) -> torch.Tensor:
        """
        Args:
            x: (B, 3, img_size, img_size) float tensor.
        Returns:
            (B, d_model) float tensor: the CLS embedding after the final LayerNorm.
            If return_all_tokens=True, returns (B, N+1, d_model).
        """
```

Your implementation should: (i) patchify the input; (ii) prepend a learnable [CLS] token; (iii) add a learnable positional embedding of shape  $(1, N + 1, d_{\text{model}})$ ; (iv) pass the sequence through `num_blocks` Transformer blocks with `is_decoder=False` and `block_size=N+1`; (v) apply a final LayerNorm; (vi) return only the [CLS] slice. (The provided Block still requires `block_size` even with `is_decoder=False`; pass `N+1`.) Store `self.d_model = d_model` and `self.num_patches = N`, since later utilities use these attributes. The §2 tests call `forward(x)` and expect the CLS embedding; in §5, you may extend the method with an optional `return_all_tokens=False` flag while keeping this default behavior.

To test your code, run `uv run pytest -k test_vit` and ensure the test passes.

## 2.4 Design Questions

### Problem (vit\_pooling): CLS token vs. mean pooling

We chose to use the [CLS] token as the image summary. An alternative is to mean-pool all patch embeddings after the final block, or to use an attention-pooling head. For downstream tasks that require *spatial* reasoning (e.g., counting objects, OCR, or visual question answering that refers to specific image regions), which pooling strategy do you expect to perform best, and why? What information is lost when we condense an entire image into a single CLS vector before passing it to a language model?

**Deliverable:** A 3–4 sentence response.

### Problem (vit\_patch\_size): Effect of patch size

Consider a  $224 \times 224$  RGB image and patch sizes  $P \in \{8, 16, 32\}$ .

- (1): Compute the number of patches  $N$  for each  $P$ . What happens to the self-attention compute cost (which scales as  $O(N^2 d_{\text{model}})$ ) as you shrink  $P$ ?
- (2): Measure forward-pass wall-clock time on a single batch of 16 images for each  $P$  using a ViT with  $d_{\text{model}} = 384$ , `num_heads` = 6, `num_blocks` = 6. Use `torch.cuda.synchronize()` around your timing block, and average over 20 steps after 5 warmup steps.
- (3): Smaller patches preserve more spatial detail but are more expensive. In one sentence, when would you accept this trade-off?

**Deliverable:** A table of times (mean  $\pm$  std) for each patch size, and a 2–3 sentence discussion.

### 3 CLIP-Style Contrastive Pretraining

Randomly initialized, your ViT produces meaningless features. To make it useful as the vision encoder for a VLM, we need to train it to produce image embeddings that capture semantic content. One powerful way to do this is the contrastive learning recipe of CLIP [2]: given a batch of (image, caption) pairs, we train the image encoder so that each image’s embedding is close to its caption’s embedding and far from every other caption’s embedding. Because captions are natural language, the resulting image embeddings inherit the semantic structure of the text encoder and can be used for zero-shot classification.

We will train on **EuroSAT** [3], a dataset of Sentinel-2 satellite images across 10 land-use classes. This gives a clean “new domain” story: off-the-shelf pretrained vision encoders see few satellite images during training, so contrastive pretraining on EuroSAT meaningfully adapts the encoder.

#### Files to update for this section

Use `vlm/data.py` and `basics/text_encoder.py` as provided. Implement projection heads and the CLIP loss in `vlm/clip.py`; implement the EuroSAT training, validation, checkpointing, and plotting workflow in `scripts/pretrain_clip.py`; use `configs/clip_eurosat.yaml` for default hyperparameters.

#### 3.1 Data and Text Encoder Setup

EuroSAT ships with class labels, not captions, so we synthesize captions from a simple template:

```
captions = [f"a satellite image of {EUROSAT_CLASSES[label]}" for label in labels]
```

For the text encoder, we use a *frozen* pretrained model so that you can focus on the image side. Use `sentence-transformers/all-MiniLM-L6-v2` (384-dim embeddings, small, fast), which is the default supported by the starter helper:

```
from basics.text_encoder import FrozenTextEncoder
text_encoder = FrozenTextEncoder("sentence-transformers/all-MiniLM-L6-v2")
text_encoder.eval() # never in train mode
with torch.no_grad():
    text_embeds = text_encoder(list_of_captions) # (B, d_text)
```

#### Problem (clip\_setup): Data pipeline and projection heads

**Files to update:** `vlm/clip.py` for `ProjectionHeads`; use `vlm/data.py` as provided.

Since the image encoder produces  $d_{\text{model}}$ -dimensional embeddings and the frozen text encoder produces  $d_{\text{text}}$ -dimensional embeddings, we need two small learnable projection heads to map both into a shared  $d_{\text{proj}}$ -dimensional space.

- (1): Use the provided `EuroSATCLIPDataset` / `build_eurosat_loaders` utilities in `vlm/data.py`. They yield (image, caption) pairs, resize images to  $64 \times 64$ , and normalize to ImageNet statistics.
- (2): Implement two `nn.Linear` projection heads in `vlm/clip.py`, `image_proj` and `text_proj`, mapping into  $d_{\text{proj}} = 256$  with no bias (as in CLIP).

(3): L2 normalization applied to both projected embeddings.

**Deliverable:** Your projection-head code. No written answer required. The unit tests focus on the contrastive loss, but these projection heads are required for the pretraining script.

### 3.2 Symmetric InfoNCE Loss

Given a batch of  $B$  image embeddings  $\mathbf{I} \in \mathbb{R}^{B \times d}$  and text embeddings  $\mathbf{T} \in \mathbb{R}^{B \times d}$  (both L2-normalized), CLIP's loss computes a similarity matrix  $\mathbf{S} = \mathbf{I}\mathbf{T}^\top / \tau$ , where  $\tau$  is a learnable temperature. The symmetric InfoNCE loss is then

$$\mathcal{L} = \frac{1}{2}(\text{CE}(\mathbf{S}, \mathbf{y}) + \text{CE}(\mathbf{S}^\top, \mathbf{y})), \quad (1)$$

where  $\mathbf{y} = (0, 1, \dots, B-1)$  are the on-diagonal indices. The first term encourages each row of  $\mathbf{S}$  (an image) to match its corresponding column (its caption); the second does the reverse.

#### Problem (infonce): Symmetric InfoNCE

**File to update:** `vlm/clip.py`.

**Deliverable:** Implement the loss function:

```
def clip_loss(
    image_embeds: torch.Tensor, # (B, d), L2-normalized
    text_embeds: torch.Tensor, # (B, d), L2-normalized
    logit_scale: torch.Tensor, # scalar, learnable, init to ln(1/0.07)
) -> torch.Tensor:
    ...
```

Following CLIP's original implementation, parameterize the inverse temperature as  $\exp(\text{logit\_scale})$ . Clamp `logit_scale` to a maximum of  $\ln(100)$  in your training loop to prevent runaway growth. To test, run `uv run pytest -k test_clip_loss`.

Include a 1–2 sentence explanation of why the loss is *symmetric* (i.e., averaged in both directions).

### 3.3 Pretraining

#### Problem (clip\_train): CLIP pretraining on EuroSAT

**Files to update:** `scripts/pretrain_clip.py`; optionally adjust `configs/clip_eurosat.yaml`.

Pretrain your ViT on EuroSAT using the contrastive objective. Recommended hyperparameters:

```
img_size = 64
patch_size = 8
d_model = 384
num_heads = 6
num_blocks = 6
```

```
batch_size = 256
lr = 3e-4
optimizer = AdamW(..., weight_decay=0.1)
num_epochs = 20
```

Split EuroSAT 80/10/10 into train/val/test. The provided loader already uses these split strings. At the end of each epoch, log the training loss and compute **zero-shot classification accuracy** on the validation set using the standard CLIP recipe: encode the 10 class-prompt captions, encode each validation image, and predict the class whose prompt has the highest cosine similarity. Because our EuroSAT captions are class-template prompts, many examples in a batch share the same caption; use the specified setup anyway, but keep this duplicate-positive issue in mind when interpreting the raw loss value.

**Deliverable:** (a) A training-loss curve, (b) a zero-shot validation accuracy curve, and (c) 2–3 sentences on how the two curves relate. Does training loss continue to improve after validation accuracy plateaus?

### Problem (clip\_zeroshot): Qualitative analysis

**Files to use:** Reuse the checkpoint and validation outputs from `scripts/pretrain_clip.py`. You may add qualitative-analysis code to that script or to a separate notebook/script for your writeup.

After training, pick 5 correctly classified and 5 incorrectly classified validation images. For each incorrectly classified image, inspect the top-3 predicted classes. Are the mistakes “reasonable” (e.g., **PermanentCrop** mistaken for **HerbaceousVegetation**) or nonsensical? What does this tell you about the structure of the learned embedding space?

**Deliverable:** 10 example images with predicted labels and a 3–4 sentence discussion.



## 4 LoRA Fine-Tuning

Full fine-tuning updates every parameter in the model, which is expensive in both memory (optimizer states) and storage (one full copy per downstream task). Low-Rank Adaptation (LoRA) [4] observes that the weight *update* during fine-tuning is often low-rank, so we can factor it as  $\Delta W = BA$  where  $A \in \mathbb{R}^{r \times d}$  and  $B \in \mathbb{R}^{d \times r}$  with  $r \ll d$ . We freeze the base weights  $W$  and only train  $A$  and  $B$ :

$$W'x = (W + \frac{\alpha}{r}BA)x. \quad (2)$$

$A$  is initialized with a Kaiming-uniform distribution and  $B$  is initialized to zero, so the adapted layer starts exactly equal to the base layer.

### Files to update for this section

Implement the LoRA modules in `basics/lora.py`. Run the RESISC45 adaptation experiments from `scripts/finetune_resisc.py`, using `configs/lora_resisc.yaml` as the default config. The LoRA tests enter through `tests/adapters.py`.

### 4.1 Implementing LoRA

#### Problem (lora\_linear): LoRA-wrapped linear layer

**File to update:** `basics/lora.py`.

**Deliverable:** Implement a module that wraps an existing `nn.Linear` layer with a LoRA adapter:

```
class LoRALinear(nn.Module):
    def __init__(self, base_layer: nn.Linear, rank: int, alpha: float):
        """
        Freeze 'base_layer', add trainable low-rank matrices A and B.
        """
        ...

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Returns base_layer(x) + (alpha / rank) * (x @ A.T @ B.T)
        ...
```

Then implement a utility `apply_lora_to_attention(model, rank, alpha)` that replaces the `q_proj` and `v_proj` linear layers inside every attention head of your ViT with `LoRALinear` wrappers (following the original LoRA paper's recommendation).

To test your code, run `uv run pytest -k test_lora_linear` and `uv run pytest -k test_apply_lora`.

Include a printout showing (i) total parameters, (ii) trainable parameters, and (iii) the ratio, for your ViT with LoRA rank 8.

### 4.2 Comparing Adaptation Strategies

To compare adaptation methods we need a *downstream* task distinct from the pretraining one. We will classify **RESISC45** [5], a different remote-sensing dataset with 45 scene categories. The

starter code provides a loader for the HuggingFace `timm/resisc45` dataset and resizes images to  $64 \times 64$  to match EuroSAT.

#### Problem (lora\_compare): Full FT vs. LoRA vs. linear probe

**Files to update:** `scripts/finetune_resisc.py`; optionally adjust `configs/lora_resisc.yaml`.

Starting from your CLIP-pretrained ViT, fine-tune on RESISC45 using each of the following strategies:

- (1): **Linear probe.** Freeze the entire ViT; train only a 45-way classification head on the CLS embedding.
- (2): **LoRA** (rank 8,  $\alpha = 16$ ). Freeze the ViT; train LoRA adapters on attention `q_proj` and `v_proj`, plus the classification head.
- (3): **Full fine-tuning.** Train every parameter.

Train each for 10 epochs with the same learning rate (you may tune per-method if results are dramatically worse; if so, report the tuning). For each, report: (a) final test accuracy, (b) number of trainable parameters, (c) peak GPU memory during training (use `torch.cuda.max_memory_allocated`), (d) wall-clock training time.

**Deliverable:** A table with these four numbers for each of the three methods, and a 4–5 sentence discussion of the trade-offs.

#### Problem (lora\_rank): Rank sweep

**Files to update:** `scripts/finetune_resisc.py`; optionally adjust `configs/lora_resisc.yaml`.

Sweep the LoRA rank  $r \in \{1, 2, 4, 8, 16, 32, 64\}$ , training each for 10 epochs on RESISC45 with  $\alpha = 2r$  (so that  $\alpha/r$  is constant). Plot test accuracy as a function of rank.

- (1): At what rank do you see diminishing returns?
- (2): How does your answer compare to the (much smaller) rank at which LoRA is typically deployed in practice (e.g.,  $r = 8$  or  $r = 16$  in large-model fine-tuning)? What does this tell you about the effective rank of the fine-tuning update?

**Deliverable:** The plot and a 3–4 sentence discussion.

## 5 Vision-Language Model

We now assemble the full Vision-Language Model. The architecture, following modern open VLMs like LLaVA [6], has three parts:

1. An **image encoder** (your CLIP-pretrained ViT from §3) that turns an image into visual features.
2. A **vision-language projector** that maps visual features into the decoder’s embedding space, producing “visual tokens”.
3. A **decoder language model** that autoregressively generates text, conditioned on both the visual tokens and any text tokens in the prompt.

### Files to update for this section

Implement the projector in `vlm/projector.py` and the fusion, injection, masking, loss-label shifting, and generation logic in `vlm/model.py`. Extend `basics/vit.py` with an optional full-token return path for all-patches and interleaved injection. Use `vlm/masking.py` as the attention-mask helper, `scripts/train_vlm.py` for CLEVR training and ablations, `scripts/eval_vlm.py` for qualitative evaluation, and `configs/vlm_clevr.yaml` for default hyperparameters.

### 5.1 Decoder Setup

We use **SmolLM2-360M-Instruct** [7] as the decoder. It is small enough to fine-tune on a single A100 but capable enough to follow instructions. Load it in `bfloat16` with `FlashAttention-2` to save memory:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

decoder = AutoModelForCausalLM.from_pretrained(
    "HuggingFaceTB/SmolLM2-360M-Instruct",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
)
tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-360M-Instruct")
```

You may use `transformers` library to *load* the model and run forward passes (via `model(inputs_embeds=...)`), but you should *not* use any training utilities (e.g., the `Trainer` class). We will construct our own training loop.

### 5.2 Dataset: CLEVR

We train and evaluate on **CLEVR** [8], a dataset of rendered 3D scenes with compositional questions (e.g., “How many red cubes are behind the blue sphere?”). CLEVR is ideal for a small VLM because (a) answers are drawn from a small vocabulary (yes/no, numbers 0–10, colors, shapes, materials), making exact-match evaluation reliable, and (b) the images are synthetic and clean, so we don’t need a heavily pretrained encoder to get useful features.

The starter code expects the preprocessed 10k-example subset at `data/clevr_mini/`. Run `uv run python scripts/download_clevr.py` to download the original-resolution CLEVR-mini zip from Google Drive and extract it. Manual link: <https://drive.google.com/file/d/1KsswLqfYL1d91pg5kGUgwt/view?usp=sharing>.

### 5.3 Vision-Language Projector

The ViT’s output embedding has dimension  $d_{\text{model}}$  (e.g., 384); the decoder expects input embeddings of dimension  $d_{\text{decoder}}$  (960 for SmolLM2-360M). We bridge them with a small MLP.

#### Problem (projector): Vision-language projector

**File to update:** `vlm/projector.py`.

**Deliverable:** Implement a 2-layer MLP projector:

```
class VisionLanguageProjector(nn.Module):
    def __init__(self, d_image: int, d_decoder: int, expansion: int = 4):
        """
        MLP: Linear(d_image, expansion * d_image) -> GELU
            -> Linear(expansion * d_image, d_decoder).
        """
        ...

    def forward(self, image_features: torch.Tensor) -> torch.Tensor:
        """
        Args:
            image_features: (B, N_vis, d_image) or (B, d_image).
        Returns:
            (B, N_vis, d_decoder) or (B, 1, d_decoder).
        """
```

The projector must handle both a single pooled image vector ( $N_{\text{vis}} = 1$ ) and a sequence of visual vectors ( $N_{\text{vis}}$  equal to the number of injected visual tokens).

Include a 1–2 sentence rationale for why we need more than a single linear layer here. Hint: think about what additional learnable capacity buys you when the encoder and decoder are kept frozen during the pretraining stage of VLM training.

### 5.4 Token Injection Strategies

A key design choice in VLMs is *how* to mix visual tokens with text tokens in the decoder’s input sequence. We will compare three strategies:

**(S1) CLS-only prefix.** Use only the ViT’s CLS embedding. The decoder sees a single visual token prepended to the text prompt:  $[\text{CLS}, t_1, t_2, \dots]$ . This is the simplest possible scheme: a single global summary of the image.

**(S2) All-patches prefix.** Use every final-layer ViT token: the CLS token plus all patch tokens. The decoder sees  $N + 1$  visual tokens prepended to the text:  $[\text{CLS}, p_1, p_2, \dots, p_N, t_1, t_2, \dots]$ . This is closer to what LLaVA-style models do than a single global image token.

**(S3) Interleaved via placeholder.** The user prompt contains a special `<image>` token. At training time, we replace that token’s embedding with the same visual-token sequence used in the all-patches prefix mode. This generalizes to multiple images per prompt.

### Problem (injection): Implementing token injection

**Files to update:** `vlm/model.py` for injection and label shifting; `basics/vit.py` for `return_all_tokens=True`.

**Deliverable:** Implement each strategy as a method on your `VisionLanguageModel` class:

```
class VisionLanguageModel(nn.Module):
    def forward(
        self,
        images: torch.Tensor,          # (B, 3, H, W)
        input_ids: torch.Tensor,       # (B, T) tokenized text
        attention_mask: torch.Tensor,  # (B, T)
        labels: torch.Tensor | None,   # (B, T) for loss computation
        injection: str = "cls",        # "cls", "all_patches", or "interleaved"
        mask_mode: str = "causal",     # "causal" or "image_bidir"
    ):
        ...
```

For the "all\_patches" and "interleaved" modes, modify your ViT from §2 to optionally return the full visual-token sequence (i.e., all  $N + 1$  tokens, including CLS) rather than only the CLS embedding.

Crucially, when labels are provided for loss computation, you must shift them to account for the injected visual tokens (the model should not be asked to *predict* visual tokens). Mask visual-token positions in the loss with `ignore_index=-100`. For answer-only VQA training, prompt/question tokens and padding should also be set to `-100`, so only answer tokens contribute to the loss.

You will compare the three strategies in the next problem.

### Problem (injection.compare): Which injection strategy is best?

**Files to update:** `scripts/train_vlm.py`; use `configs/vlm_clevr.yaml` for the shared defaults.

Train a VLM with each of the three injection strategies for 2000 steps on CLEVR. For each, keep the ViT and decoder frozen, and train only the projector. Use batch size 32, learning rate  $1 \times 10^{-4}$ , and the AdamW optimizer.

For each strategy, report:

- (1): Validation exact-match accuracy on 500 held-out CLEVR examples.
- (2): Number of visual tokens injected per example.
- (3): Peak GPU memory during training.
- (4): Wall-clock time per step.

**Deliverable:** A 3-row table and a 1-paragraph discussion. Which strategy gives the best accuracy, and is the extra cost worth it? You should observe a clear connection to the CLS-vs-patch pooling question from Problem (vit\_pooling).

## 5.5 Attention Masking

The decoder is causal: each text token attends only to previous tokens. But how should attention work *inside* the visual prefix? Two reasonable choices:

**(M1) Fully causal.** Even among visual tokens, patch  $i$  only attends to patches  $1, \dots, i$ . This is the simplest thing to implement — just treat visual tokens as ordinary tokens — but it imposes an arbitrary ordering on the 2D grid.

**(M2) Bidirectional inside image, causal across boundary.** Visual tokens attend to each other without restriction; text tokens attend causally to all prior text tokens and to all visual tokens. This preserves the ViT’s bidirectional view of the image.

### Problem (masking): Image-block attention

**Files to update:** `vlm/model.py` to select the mask; use `vlm/masking.py` as the provided helper. Run the comparison through `scripts/train_vlm.py`.

- (1): Draw the attention mask for a sequence of 4 visual tokens followed by 3 text tokens, under each of (M1) and (M2). Use a  $7 \times 7$  grid with shaded cells for allowed positions.
- (2): Which of (M1) and (M2) do you expect to perform better, and why?
- (3): Modify your VLM to support both masks (the starter code provides a utility for constructing a custom 4D attention mask to pass into `SmolLM2`). Briefly train with each for 500 steps on CLEVR (using the all-patches injection from Problem (injection\_compare)) and report validation accuracy.

**Deliverable:** Two mask diagrams, a 3–4 sentence discussion, and a 2-row table of validation accuracies.

## 5.6 Freezing Strategies

In the problems above, we froze the encoder and decoder and trained only the projector. This is the *pretraining* stage of the modern two-stage recipe; the *instruction-tuning* stage then unfreezes additional parameters.

### Problem (freezing): What to train, and when

**File to update:** `scripts/train_vlm.py`.

Starting from the best injection + masking configuration from the previous problems, run four training configurations for 1500 steps each:

| Configuration               | Encoder | Projector | Decoder       |
|-----------------------------|---------|-----------|---------------|
| A: projector only           | frozen  | trained   | frozen        |
| B: projector + decoder LoRA | frozen  | trained   | LoRA (rank 8) |
| C: projector + full decoder | frozen  | trained   | full FT       |
| D: all three                | full FT | trained   | full FT       |

For configuration B, note that the §4 helper `apply_lora_to_attention` only targets `basics.model.Head` instances, so decoder LoRA should wrap `SmolLM2 q_proj/v_proj` linear layers directly with `LoRALinear`.

Report validation exact-match accuracy, trainable parameter count, and peak memory for each. Which configuration gives the best trade-off between accuracy and cost? Discuss in the context of the two-stage (pretraining, instruction-tuning) recipe.

**Deliverable:** A 4-row table and a 5–6 sentence discussion.

## 5.7 Qualitative Evaluation

**Problem (vlm\_qualitative):** What has your VLM learned?

**File to update:** `scripts/eval_vlm.py`.

Take your best VLM and generate responses on 10 held-out CLEVR examples. Include the image, the question, the ground-truth answer, and your model’s generation. Pick a mix of correct and incorrect cases. For each incorrect case, hypothesize whether the failure is in the encoder (image not well understood) or the decoder (language component misinterpreting the question). How would you design an experiment to distinguish between these two failure modes?

**Deliverable:** 10 example rows and a 4–5 sentence discussion.

## 6 Positional Encodings and RoPE

Up to this point, our ViT has used *learned* positional embeddings — a fixed  $(N+1) \times d_{\text{model}}$  parameter matrix added to the input. This works, but it has two disadvantages: (1) it can't extrapolate to sequences longer than those seen at training time, and (2) it bakes positional information into the input, rather than the attention computation itself.

**Rotary Position Embeddings (RoPE)** [9] avoid both issues. The idea is to rotate each query and key vector by an angle that depends on its position, *inside* the attention computation. After rotation, the dot product  $\mathbf{q}_m \cdot \mathbf{k}_n$  depends only on the relative offset  $m - n$  (not on absolute  $m$  or  $n$ ), giving the model a natural inductive bias for relative position. RoPE is what modern decoders (SmolLM2, Qwen, Llama, ...) use in practice.

### Files to update for this section

Implement RoPE1D and RoPE2D in `basics/rope.py`; the tests call them through `tests/adapters.py`. For learned-vs-RoPE ViT ablations, update `basics/vit.py` to support the positional-encoding choice and run the retraining/evaluation through `scripts/pretrain_clip.py`. The bonus M-RoPE work belongs in `vlm/model.py`, with training/evaluation support in `scripts/train_vlm.py` and `scripts/eval_vlm.py`.

### 6.1 1D RoPE

For a  $d$ -dimensional vector  $\mathbf{x}$  at position  $m$ , 1D RoPE groups the  $d$  dimensions into  $d/2$  pairs and rotates each pair by angle  $m\theta_i$ , where  $\theta_i = \text{base}^{-2i/d}$  for  $i = 0, \dots, d/2 - 1$  (the base is typically  $10^4$ ).

Concretely, for a pair  $(x_{2i}, x_{2i+1})$ ,

$$\text{RoPE}_m(x_{2i}, x_{2i+1}) = (x_{2i} \cos(m\theta_i) - x_{2i+1} \sin(m\theta_i), x_{2i} \sin(m\theta_i) + x_{2i+1} \cos(m\theta_i)). \quad (3)$$

We apply RoPE to queries and keys (not values) inside each attention head before computing  $\mathbf{q}\mathbf{k}^\top$ .

#### Problem (rope\_1d): Implementing 1D RoPE

**File to update:** `basics/rope.py`.

**Deliverable:** Implement RoPE as a module:

```
class RoPE1D(nn.Module):
    def __init__(self, head_dim: int, max_seq_len: int, base: float =
        10000.0):
        ...

    def forward(self, x: torch.Tensor, positions: torch.Tensor) -> torch.
        Tensor:
        """
        Args:
            x: (B, num_heads, T, head_dim).
            positions: (T,) integer positions.
        Returns:
            (B, num_heads, T, head_dim): x with RoPE applied.
        """
```



Precompute  $\cos(m\theta_i)$  and  $\sin(m\theta_i)$  up to `max_seq_len` in `__init__` and register them as buffers (not parameters). To test, run `uv run pytest -k test_rope_1d`. Also verify manually that applying RoPE preserves the norm of each vector (up to numerical precision), and report what you measure.

### Problem (rope\_vs\_learned): Learned PE vs. RoPE in the ViT

**Files to update:** `basics/vit.py` for the ViT positional-encoding option; `scripts/pretrain_clip.py` for the retraining and extrapolation evaluation.

Modify your ViT to use RoPE on  $(\mathbf{q}, \mathbf{k})$  inside attention, instead of adding learned positional embeddings to the input. The patch positions are simply their 1D index (we will make this more clever in the next problem).

Retrain CLIP-style on EuroSAT for 20 epochs using (a) learned PE and (b) 1D RoPE. Report zero-shot validation accuracy for each.

Then perform a **length-extrapolation test**: evaluate both models on EuroSAT images upsampled to  $96 \times 96$  (keeping the same patch size 8), which produces 144 patches instead of the 64 seen at training. For the learned-positional-embedding baseline, interpolate the learned patch positional embeddings from the  $8 \times 8$  training grid to the  $12 \times 12$  evaluation grid before adding them to the patch tokens; keep the CLS positional embedding separate. How does each model's accuracy degrade?

**Deliverable:** A 2-row table (one per PE method) with two columns (train-size accuracy and extrapolated-size accuracy), and a 3–4 sentence discussion of the extrapolation behavior.

## 6.2 2D RoPE for Image Patches

Treating image patches as a 1D sequence discards the fact that they live on a 2D grid. A patch at grid position  $(x, y)$  has *two* meaningful coordinates, and a well-designed positional encoding should reflect this.

A clean extension: split the head dimension  $d$  in half. Apply 1D RoPE to the first  $d/2$  dimensions using the patch's  $x$ -coordinate, and apply 1D RoPE to the second  $d/2$  dimensions using the  $y$ -coordinate. After rotation, the dot product between two patches depends on their 2D *relative* offset.

### Problem (rope\_2d): 2D RoPE

**Files to update:** `basics/rope.py` for RoPE2D; `basics/vit.py` for using 2D patch coordinates in the ViT; `scripts/pretrain_clip.py` for the ablation runs.

**Deliverable:** Implement RoPE2D:

```
class RoPE2D(nn.Module):
    def __init__(self, head_dim: int, grid_size: int, base: float = 10000.0)
        :
        # head_dim must be divisible by 4
        # (splits into 2D x/y, each with real/imaginary parts).
        ...
```

```
def forward(
    self,
    x: torch.Tensor,
    x_coords: torch.Tensor,
    y_coords: torch.Tensor,
) -> torch.Tensor:
    ...
```

Swap RoPE1D for RoPE2D in your ViT (using each patch’s  $(x, y)$  grid indices) and re-run the CLIP pretraining + zero-shot evaluation from Problem (rope\_vs\_learned). Does 2D RoPE improve over 1D RoPE on EuroSAT? Include the length-extrapolation test with 2D RoPE as well.

Provide your implementation, zero-shot accuracy, and a 2–3 sentence discussion.

### 6.3 Multimodal RoPE (M-RoPE)

When the decoder in our VLM is fed a sequence like  $[\text{CLS}, p_1, p_2, \dots, p_N, t_1, t_2, \dots]$ , a subtle question arises: what *position IDs* should the visual tokens have? The naive answer is  $(0, 1, 2, \dots, N)$  for the visual tokens and  $(N + 1, N + 2, \dots)$  for text tokens — but this treats the image as a 1D sequence (losing 2D structure) and artificially pushes text tokens to high positions (possibly outside the range the decoder was trained on).

Qwen2-VL [10] proposes **M-RoPE**, which assigns each token a 3D position  $(t, x, y)$ : a temporal index, a horizontal index, and a vertical index. For text tokens, all three coordinates advance together. For image tokens, the temporal index is fixed, and the  $x$  and  $y$  coordinates encode the 2D patch grid. RoPE is then applied separately to three chunks of the head dimension.

#### Problem (mrope\_written): Reasoning about M-RoPE

**Files to update:** None; this is a written-response problem.

Answer the following in prose (no coding required):

- (1): What goes wrong with naive 1D position IDs  $(0, 1, 2, \dots)$  when we inject 64 patch tokens plus a CLS token before a 50-token text prompt? Think about (a) position-ID values the decoder was trained on, and (b) the 2D structure of the image.
- (2): Under M-RoPE, what position does the first text token get (as a function of the image’s grid size)? Why is this choice sensible?
- (3): Why does M-RoPE split the head dimension into three chunks rather than two? What would break if we only used  $(x, y)$  and dropped the temporal  $t$ ?

**Deliverable:** A 1-paragraph response to each subquestion (3 paragraphs total).

#### Problem (mrope\_impl): Implementing M-RoPE (bonus)

**Files to update:** `vlm/model.py` for M-RoPE-style position assignment; `scripts/train_vlm.py` and `scripts/eval_vlm.py` for the bonus comparison.

Implement an M-RoPE-style position assignment for your VLM. Retrain the best configuration from §5 for 1500 steps, using (a) naive 1D position IDs and (b) M-RoPE-style position IDs. Does M-RoPE improve CLEVR accuracy? Does it help more on questions that refer to spatial relations (“left of”, “behind”, “in front of”) than on other questions?

**Deliverable:** A 2-row table with overall accuracy and spatial-question accuracy, plus a 3–4 sentence discussion.

## References

- [1] Dosovitskiy, Alexey, et al. “An image is worth 16x16 words: Transformers for image recognition at scale.” *ICLR*, 2021. <https://arxiv.org/abs/2010.11929>.
- [2] Radford, Alec, et al. “Learning transferable visual models from natural language supervision.” *ICML*, 2021. <https://arxiv.org/abs/2103.00020>.
- [3] Helber, Patrick, et al. “EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification.” *IEEE JSTARS*, 2019. <https://arxiv.org/abs/1709.00029>.
- [4] Hu, Edward J., et al. “LoRA: Low-rank adaptation of large language models.” *ICLR*, 2022. <https://arxiv.org/abs/2106.09685>.
- [5] Cheng, Gong, Junwei Han, and Xiaoqiang Lu. “Remote sensing image scene classification: Benchmark and state of the art.” *Proceedings of the IEEE*, 2017. <https://arxiv.org/abs/1703.00121>.
- [6] Liu, Haotian, et al. “Visual instruction tuning.” *NeurIPS*, 2023. <https://arxiv.org/abs/2304.08485>.
- [7] Allal, Loubna Ben, et al. “SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model.” arXiv preprint arXiv:2502.02737, 2025. <https://arxiv.org/abs/2502.02737>.
- [8] Johnson, Justin, et al. “CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning.” *CVPR*, 2017. <https://arxiv.org/abs/1612.06890>.
- [9] Su, Jianlin, et al. “RoFormer: Enhanced transformer with rotary position embedding.” *Neurocomputing*, 2024. <https://arxiv.org/abs/2104.09864>.
- [10] Wang, Peng, et al. “Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution.” arXiv preprint arXiv:2409.12191, 2024. <https://arxiv.org/abs/2409.12191>.